

Dokumentáció: Mozi Jegyfoglaló Mikroszolgáltatás-rendszer

DENGEL XÉNIA ESZTER
QVDU2F
2025/26/2

1. Bevezetés és Célkitűzés

A feladatkiírásnak megfelelően a projekt célja egy modern, konténerizált mozi jegyfoglaló rendszer kidolgozása, amely mikroszolgáltatás-architektúrára épül. Azért választottam ezt a témakört, mert a jegyértékesítési folyamatok kiválóan modellezik a valós idejű adatkezelés (Redis), az aszinkron üzenetküldés (RabbitMQ) és a skálázhatóság kihívásait. A projekt nem csupán egy webes felületet kínál, hanem egy teljes ökoszisztémát, amely magában foglalja a háttérfolyamatok automatizálását, a látogatottsági adatok perzisztenciáját és a rendszer teljes körű nyomonkövethetőségét (Jaeger).

2. Alkalmazott technológiák és felépítés

Az alkalmazás több egymással kommunikáló komponensből áll, melyeket Docker környezetben izoláltam:

- **Frontend & API:** Flask (Python) alapú webalkalmazás modern, sötét tónusú felhasználói felülettel.
- **Adatkezelés:** Redis kulcs-érték tároló a valós idejű látogatásszámláláshoz.
- **Üzenetküldés:** RabbitMQ bróker, amely a jegyvásárlási kéréseket sorolja be és továbbítja.
- **Háttérfeldolgozás:** Egy dedikált Worker (Consumer), amely aszinkron módon dolgozza fel a beérkező vásárlásokat.
- **Megfigyelhetőség:** OpenTelemetry és Jaeger integráció a kérések életútjának (Distributed Tracing) vizualizálására.
- **CI/CD:** GitHub Actions munkafolyamat, amely minden feltöltésnél automatikusan validálja a rendszer építhetőségét.

3. Fejlesztés során tapasztalt kihívások

Komponensek szinkronizációja: A legnagyobb kihívást a konténerek indulási sorrendje jelentette. Meg kellett oldanom, hogy a Worker ne induljon el hamarabb, mint ahogy a RabbitMQ üzemkész állapotba kerül. Ezt Python oldali késleltetéssel (retry logic) és a Docker-compose konfigurálásával sikerült stabilizálni.

CI/CD integráció: A GitHub Actions környezetben a Docker-compose alapú build folyamatok beállítása precíz konfigurációt igényelt, különös tekintettel a függőségek (requirements.txt) kezelésére és a YAML szintaxis pontosságára.

4. Összegzés és konklúzió

A Python nyelv és a Docker ökoszisztéma rendkívül hatékony eszköznek bizonyult egy komplex, elosztott rendszer létrehozására. A projekt igazolta, hogy a mikroszolgáltatások használatával a webalkalmazások komponensei (adatbázis, üzenetsor, frontend) szétválaszthatók, így a rendszer rugalmasabbá és könnyebben karbantarthatóvá válik.

A fejlesztés legfőbb eredménye egy stabil, skálázható architektúra, amely automatizált tesztelési folyamattal (CI/CD) rendelkezik, és megfelel a modern szoftverfejlesztési elvárásoknak.

A projekt forráskódja, a Docker konfiguráció és a prezentációs videó elérhetősége:

- GitHub Repozitórium: <https://github.com/Xenon066/webprojekt>
- Elevator Pitch videó (OneDrive): [Webprogramozas_QVDU2F](#)